

Interaktive Lernplattform für Grundlagen Informatik

Projektarbeit – SS 26

31. Juli 2026

Studenten:	Markus Nöth	markus.noeth@study.thws.de	5123027
	Richard Schröder	richard.schroeder@study.thws.de	5123042
	Enrico Ebert	enrico.ebert@study.thws.de	5123098
	Duy Tan Pham	duytan.pham@study.thws.de	5123063
	Kristian Popov	kristian.popov@study.thws.de	5123029

Dozent: Prof. Dr. Peter Braun

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Problemstellung (Author) . . . 1
1.2 Zielsetzung der Arbeit (Author) . . . 1
1.3 Methodik (Author) . . . 1

2 Übergreifende Architektur & Systemdesign **1**
2.1 Technologiestack (Author) . . . 1
2.2 Modulare Softwarearchitektur (Author) . . . 1
2.3 Unified UI-Konzept (Author) . . . 1
2.4 Simulator-Architektur (Author) . . . 1
2.5 Datenmodellierung (Author) . . . 1

3 Einsatz von Künstlicher Intelligenz im Projekt **1**
3.1 KI Reflexion (Markus Nöth) . . . 1
3.1.1 KI als Co-Developer (Markus Nöth) . . . 2
3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth) . . . 2
3.1.3 Kritische Reflexion (Markus Nöth) . . . 2
3.2 KI Reflexion (Richard Schröder) . . . 2
3.2.1 KI als Co-Developer (Richard Schröder) . . . 2
3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder) . . . 2
3.2.3 Kritische Reflexion (Richard Schröder) . . . 2
3.3 KI Reflexion (Enrico Ebert) . . . 2
3.3.1 KI als Co-Developer (Enrico Ebert) . . . 2
3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert) . . . 2
3.3.3 Kritische Reflexion (Enrico Ebert) . . . 2
3.4 KI Reflexion (Duy Tan Pham) . . . 2
3.4.1 KI als Co-Developer (Duy Tan Pham) . . . 2
3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham) . . . 3
3.4.3 Kritische Reflexion (Duy Tan Pham) . . . 3
3.5 KI Reflexion (Kristian Popov) . . . 3
3.5.1 KI als Co-Developer (Kristian Popov) . . . 3
3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov) . . . 4
3.5.3 Kritische Reflexion (Kristian Popov) . . . 4

4 Designentscheidungen der Fachmodule **5**
4.1 Turing Maschine (Author) . . . 5
4.1.1 Entscheidungen in der Architektur (Author) . . . 5
4.1.2 Entscheidungen im UI (Author) . . . 5
4.1.3 Andere Entscheidungen (Author) . . . 5
4.2 Endliche Automaten (Author) . . . 5
4.2.1 Entscheidungen in der Architektur (Author) . . . 5
4.2.2 Entscheidungen im UI (Author) . . . 5
4.2.3 Andere Entscheidungen (Author) . . . 5
4.3 Reguläre Sprachen (Author) . . . 5
4.3.1 Entscheidungen in der Architektur (Author) . . . 5
4.3.2 Entscheidungen im UI (Author) . . . 5
4.3.3 Andere Entscheidungen (Author) . . . 5
4.4 Zahlensysteme (Author) . . . 5
4.4.1 Entscheidungen in der Architektur (Author) . . . 5

4.4.2 Entscheidungen im UI (Author) . . . 5
4.4.3 Andere Entscheidungen (Author) . . . 5
4.5 Moore & Mealy Automaten (Author) . . . 6
4.5.1 Entscheidungen in der Architektur (Author) . . . 6
4.5.2 Entscheidungen im UI (Author) . . . 6
4.5.3 Andere Entscheidungen (Author) . . . 6
4.6 Disjunktive Normalform (Author) . . . 6
4.6.1 Entscheidungen in der Architektur (Author) . . . 6
4.6.2 Entscheidungen im UI (Author) . . . 6
4.6.3 Andere Entscheidungen (Author) . . . 6
4.7 Encoding (Author) . . . 6
4.7.1 Entscheidungen in der Architektur (Author) . . . 6
4.7.2 Entscheidungen im UI (Author) . . . 6
4.7.3 Andere Entscheidungen (Author) . . . 6
4.8 Java (Author) . . . 6
4.8.1 Entscheidungen in der Architektur (Author) . . . 6
4.8.2 Entscheidungen im UI (Author) . . . 6
4.8.3 Andere Entscheidungen (Author) . . . 6

5 Evaluation & Projekterfahrung 6
5.1 Qualitätssicherung (Author) . . . 6
5.2 Abgleich mit dem Lastenheft (Author) . . . 6
5.3 Lessons Learned (Author) . . . 6
5.4 Design-Iterationen (Author) . . . 7

6 Fazit & Ausblick 7
6.1 Zusammenfassung der Projektergebnisse (Author) . . . 7
6.2 Perspektiven (Author) . . . 7

Anhang i
Abkürzungsverzeichnis . . . i
Literatur . . . i

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Eine Arbeit von Technische Hochschule Würzburg-Schweinfurt (THWS) Studenten.

1.1 Problemstellung (Author)

Die Hürde der Abstraktion in Grundlagen Informatik

1.2 Zielsetzung der Arbeit (Author)

Didaktische Aufbereitung komplexer Modelle durch interaktive Simulationen

1.3 Methodik (Author)

Vorgehensweise im Team und Rollenverteilung

2 Übergreifende Architektur & Systemdesign

2.1 Technologiestack (Author)

React 19, TypeScript, Vite, Tailwind CSS 4, shadcn/ui, Zustand, i18next, Lucide Icons

2.2 Modulare Softwarearchitektur (Author)

Single Source of Truth (`moduleCatalog.tsx`), lazy Loading, Shared Topic Pattern – Sicherstellung der Erweiterbarkeit für neue Themen

2.3 Unified UI-Konzept (Author)

Wiederverwendbare Shared Components, konsistentes Designsystem (CSS-Variablen, Dark Mode, moduleigene Akzentfarben, Geist Font)

2.4 Simulator-Architektur (Author)

Das `ISimulator<TConfig, TState>`-Interface und der generische `useSimulationEngine`-Hook als Grundlage aller schrittweisen Simulationen

2.5 Datenmodellierung (Author)

Import/Export über `ISimulatorTransferCodec` mit Base64-Kodierung für Vorlesungskompatibilität

3 Einsatz von Künstlicher Intelligenz im Projekt

3.1 KI Reflexion (Markus Nöth)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.1.1 KI als Co-Developer (Markus Nöth)

Reflektieren und so...

3.1.2 Automatisierte Aufgabengenerierung (Markus Nöth)

Reflektieren und so...

3.1.3 Kritische Reflexion (Markus Nöth)

Reflektieren und so...

3.2 KI Reflexion (Richard Schröder)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.2.1 KI als Co-Developer (Richard Schröder)

Reflektieren und so...

3.2.2 Automatisierte Aufgabengenerierung (Richard Schröder)

Reflektieren und so...

3.2.3 Kritische Reflexion (Richard Schröder)

Reflektieren und so...

3.3 KI Reflexion (Enrico Ebert)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.3.1 KI als Co-Developer (Enrico Ebert)

Reflektieren und so...

3.3.2 Automatisierte Aufgabengenerierung (Enrico Ebert)

Reflektieren und so...

3.3.3 Kritische Reflexion (Enrico Ebert)

Reflektieren und so...

3.4 KI Reflexion (Duy Tan Pham)

Nutzung von Gemini 3.1 Pro, ChatGPT und Codex als Entwicklungswerkzeuge – Effizienzsteigerung bei der Implementierung mathematischer Logik und UI-Komponenten

3.4.1 KI als Co-Developer (Duy Tan Pham)

Reflektieren und so...

3.4.2 Automatisierte Aufgabengenerierung (Duy Tan Pham)

Reflektieren und so...

3.4.3 Kritische Reflexion (Duy Tan Pham)

Reflektieren und so...

3.5 KI Reflexion (Kristian Popov)

Mein Schwerpunkt lag während des Projekts auf den Modulen *Endliche Automaten* und *Reguläre Sprachen*. Ausprobiert habe ich dabei mehrere Modelle: Gemini 3.1 Pro, die kostenlose Version von ChatGPT sowie Claude Opus 4.8 und später Opus 5.

Dabei zeigte sich, dass die Modelle unterschiedliche Stärken haben. Einige erklärten theoretische Zusammenhänge verständlicher, andere halfen zuverlässiger bei der Implementierung und beim Auffinden von Fehlern. Überwiegend habe ich mit Claude gearbeitet, weil die Antworten bei Programmieraufgaben konsistenter ausfielen und ich weniger nachbessern musste.

Ein Unterschied, der mir erst mit der Zeit bewusst wurde, betrifft den Aufwand, den ein Modell in eine einzelne Antwort investiert. Viele Modelle lassen sich so einstellen, dass sie vor der eigentlichen Antwort mehr oder weniger ausführlich planen. Ein höherer Aufwand bedeutet mehr verbrauchte Tokens, längere Wartezeit und höhere Kosten, führt aber nicht automatisch zu einem besseren Ergebnis. Für kleine, klar umrissene Änderungen war die niedrige Stufe ausreichend und schlicht schneller. Sobald eine Aufgabe mehrere Dateien betraf oder eine Abwägung zwischen mehreren Lösungswegen verlangte, lieferte erst die höhere Stufe brauchbare Vorschläge. Diese Wahl bewusst zu treffen, gehörte für mich zum Umgang mit den Werkzeugen dazu.

Eingesetzt habe ich die KI vor allem, um Lösungsansätze durchzusprechen, Fehler einzugrenzen und einzelne Codeabschnitte zu überarbeiten. Geprüft, getestet und an die Anforderungen des Projekts angepasst habe ich die Vorschläge selbst; die Verantwortung für die technischen Entscheidungen und die finale Implementierung lag durchgehend bei mir.

3.5.1 KI als Co-Developer (Kristian Popov)

Die KI habe ich nicht über eine Chat-Oberfläche bedient, sondern direkt in die Entwicklungsumgebung eingebunden. Codeausschnitte mussten damit nicht mehr von Hand in ein zweites Fenster übertragen werden: Das Modell las die Projektdateien selbst, schlug Änderungen vor und konnte deren Wirkung unmittelbar im Projekt prüfen.

Aktiv waren mehrere Erweiterungen aus dem offiziellen Verzeichnis `claude-plugins-official`. Für meine Module waren vor allem zwei davon relevant:

`typescript-lsp`
: bindet den Sprachserver von TypeScript über das Language Server Protocol (LSP) ein. Der KI stehen dadurch dieselben Typinformationen, Definitionen und Referenzen zur Verfügung wie der Entwicklungsumgebung. Bezeichner und Signaturen musste das Modell also nicht aus dem Quelltext erschließen, sondern konnte gezielt danach suchen; Änderungen blieben über Dateigrenzen hinweg konsistent.

`chrome-devtools-mcp`
: verbindet die KI über das Model Context Protocol (MCP) mit den Entwicklerwerkzeugen des Browsers. Konsolenausgaben und das Verhalten der laufenden Anwendung ließen sich damit direkt auswerten. Bei der Fehlersuche in den Simulatoren war das der praktischste

Gewinn: Ein fehlerhafter Zustandsübergang wurde am tatsächlichen Ablauf sichtbar und musste nicht aus dem Quelltext rekonstruiert werden.

Zusätzlich habe ich das Projekt einmalig mit Graphify [1] analysiert und daraus einen Wissensgraphen erzeugt. Graphify verbindet eine statische Analyse auf Basis von Tree-sitter mit einer semantischen Auswertung durch ein Sprachmodell und erfasst neben dem Quelltext auch Dokumentation und Diagramme. Die Knoten des Graphen stehen für Dateien, Klassen und Funktionen, die Kanten für deren Beziehungen untereinander. Zu Beginn einer Sitzung musste die KI das Projekt dadurch nicht erneut vollständig einlesen, sondern konnte den für die jeweilige Aufgabe relevanten Teilgraphen abfragen. Das sparte Zeit und hielt vor allem den Kontext über mehrere Entwicklungssitzungen hinweg stabil.

Ein eigener Punkt war die Konfiguration der KI. Anstatt die Modelle frei entscheiden zu lassen, habe ich sie über dauerhaft im Projekt hinterlegte Anweisungen dazu verpflichtet, bei Architektur- und Implementierungsfragen zuerst mehrere Alternativen vorzuschlagen und zu jeder davon die Vor- und Nachteile sowie die Gründe für ihren Einsatz zu benennen. Weil diese Vorgaben zum Projekt gehören, galten sie in jeder Sitzung, ohne dass ich sie erneut formulieren musste. Die Auswahl habe ich anschließend selbst getroffen. Das kostet pro Entscheidung mehr Zeit, zwingt aber zu einer bewussten Abwägung und verhindert, dass ein Vorschlag ungeprüft im Projekt landet.

3.5.2 Automatisierte Aufgabengenerierung (Kristian Popov)

Bei wiederkehrenden Arbeiten hat die KI am meisten Zeit gespart. Dazu zählten Refactorings und Optimierungen am Quellcode, das Erzeugen von Boilerplate-Code, das Schreiben von Unit-Tests sowie die Internationalisierung der Benutzeroberfläche, die im Projekt zweisprachig in Deutsch und Englisch vorliegt.

Am klarsten gelohnt hat sich der Einsatz bei den Tests. Für die Automatenlogik, das Parsen der Grammatiken und die Validierung der Programme ließen sich aus der vorhandenen Implementierung zügig Testfälle ableiten, auch für Eingaben, an die ich beim Schreiben des Codes nicht gedacht hatte.

Hilfreich war sie außerdem beim Aufräumen: TypeScript-Warnungen aufspüren, Benennungen vereinheitlichen, Kommentare und Methodendokumentation vorformulieren. Wörtlich übernehmen ließ sich davon allerdings wenig, weil den Vorschlägen der Projektkontext fehlte; als Ausgangspunkt waren sie trotzdem brauchbar.

3.5.3 Kritische Reflexion (Kristian Popov)

Die Vorschläge der Modelle waren häufig brauchbar, aber nicht immer korrekt oder auf die bestehende Projektarchitektur abgestimmt. Wiederholt kamen APIs zum Einsatz, die in der verwendeten Version so nicht mehr existierten. Randfälle blieben unberücksichtigt, und einzelne Änderungen wirkten sich an Stellen aus, die im Vorschlag selbst gar nicht erwähnt waren. Gerade bei formalen Sprachen ist „plausibel“ etwas anderes als „korrekt“: Der erzeugte Code sieht wohlgeformt aus, und ob er das geforderte Verhalten tatsächlich abbildet, zeigt sich erst beim Durchspielen konkreter Eingaben.

Das zweite Muster betraf die Struktur. Da sich *Endliche Automaten* und *Reguläre Sprachen* stark ähneln, duplizierte die KI bestehende Implementierungen, statt sie zu abstrahieren. So entstanden Simulator- und Übersichtsseiten, die sich zwischen den Modulen kaum unterschieden, und Funktionen, die nie in der Oberfläche ankamen und später wieder entfernt werden mussten. Betraf eine Änderung beide Module zugleich, etwa die einheitliche Gruppierung von Beispielen

und Übungen nach Kategorien, reichte der Überblick des Modells zudem nicht aus, um alle betroffenen Stellen zu erfassen. Solche Aufgaben musste ich in kleinere Schritte zerlegen.

Unter dem Strich hat die KI die Entwicklung spürbar beschleunigt, vor allem beim Ausbauen bereits verstandener Lösungen. Für den fachlichen Kern blieb sie ein Vorschlagsgeber, dessen Ergebnisse ich gegen die Theorie und gegen die laufende Anwendung prüfen musste; der eingesparte Implementierungsaufwand verschiebt sich damit zum Teil in Prüfaufwand. Interessant wäre gewesen, dem mit Sakana Fugu zu begegnen, einem Orchestrator, der je nach Aufgabe unterschiedliche Modelle auswählt [2]. Das System erschien allerdings erst kurz vor Abschluss meiner Arbeit, sodass ich es nicht mehr einsetzen konnte.

4 Designentscheidungen der Fachmodule

4.1 Turing Maschine (Author)

4.1.1 Entscheidungen in der Architektur (Author)

4.1.2 Entscheidungen im UI (Author)

4.1.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.2 Endliche Automaten (Author)

4.2.1 Entscheidungen in der Architektur (Author)

4.2.2 Entscheidungen im UI (Author)

4.2.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.3 Reguläre Sprachen (Author)

4.3.1 Entscheidungen in der Architektur (Author)

4.3.2 Entscheidungen im UI (Author)

4.3.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.4 Zahlensysteme (Author)

4.4.1 Entscheidungen in der Architektur (Author)

4.4.2 Entscheidungen im UI (Author)

4.4.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.5 Moore & Mealy Automaten (Author)

4.5.1 Entscheidungen in der Architektur (Author)

4.5.2 Entscheidungen im UI (Author)

4.5.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.6 Disjunktive Normalform (Author)

4.6.1 Entscheidungen in der Architektur (Author)

4.6.2 Entscheidungen im UI (Author)

4.6.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.7 Encoding (Author)

4.7.1 Entscheidungen in der Architektur (Author)

4.7.2 Entscheidungen im UI (Author)

4.7.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

4.8 Java (Author)

4.8.1 Entscheidungen in der Architektur (Author)

4.8.2 Entscheidungen im UI (Author)

4.8.3 Andere Entscheidungen (Author)

Die Punkte sind nur Vorschläge ...

5 Evaluation & Projekterfahrung

5.1 Qualitätssicherung (Author)

CI/CD-Pipeline (GitLab Runner auf Raspberry Pi), automatisiertes Linting, Formatierung (Prettier), Unit-Tests (Vitest) und E2E-Tests (Playwright)

5.2 Abgleich mit dem Lastenheft (Author)

Wurden die Qualitätsziele (Performance, Usability, Barrierefreiheit durch Dark Mode und i18n) erreicht?

5.3 Lessons Learned (Author)

Herausforderungen in der Team-Kollaboration (Git-Workflow, Merge-Requests, geteilte Architektur) und technische Hürden

5.4 Design-Iterationen (Author)

Welche Entscheidungen mussten im Prozess revidiert werden?

6 Fazit & Ausblick

6.1 Zusammenfassung der Projektergebnisse (Author)

Sieben interaktive Fachmodule in zwei Lernbereichen

6.2 Perspektiven (Author)

Gamification (Badges/Punkte), LMS-Anbindung (Moodle/ILIAS) und Mobile Learning (responsive Optimierung)

Anhang

Abkürzungsverzeichnis

LSP Language Server Protocol; Protokoll zwischen Editor und Sprachserver für Codenavigation, Typinformationen und Refactoring

MCP Model Context Protocol; Offener Standard, über den Sprachmodelle externe Werkzeuge und Datenquellen anbinden

THWS Technische Hochschule Würzburg-Schweinfurt; Hochschule aus Würzburg und Schweinfurt (Bayern) [3]

Literatur

[1] S. Shamsi. „Graphify - Multi-modal Knowledge Graph Builder," besucht am 31. Juli 2026. Adresse: `https://graphify.net/`

[2] Sakana AI. „Sakana Fugu - Multi-agent System as A Model," besucht am 31. Juli 2026. Adresse: `https://sakana.ai/fugu/`

[3] Technische Hochschule Würzburg-Schweinfurt. „THWS - Technische Hochschule Würzburg-Schweinfurt," besucht am 14. Juni 2026. Adresse: `https://thws.de`